

Guide de Deployment — Epicerie Africaine

Version : 1.0 **Date :** Fevrier 2026 **Auteur :** Equipe technique **Confidentialite :** Document interne — Ne pas partager avec le client final

Table des matieres

1. [Prerequis techniques](#)
2. [Architecture du projet](#)
3. [Installation locale](#)
4. [Configuration Sanity \(CMS\)](#)
5. [Configuration Supabase \(Auth + BDD\)](#)
6. [Configuration Stripe \(Paielements\)](#)
7. [Configuration Cal.com \(Rendez-vous\)](#)
8. [Alimenter le catalogue \(Sanity Studio\)](#)
9. [Configuration Admin Hub](#)
10. [Securite](#)
11. [Tests fonctionnels](#)
12. [Deployment en production](#)
13. [Personnalisation pour un nouveau client](#)
14. [Maintenance et operations courantes](#)
15. [Checklist de livraison](#)
16. [Depannage \(FAQ\)](#)

1. Prerequis techniques

Logiciels requis

Logiciel	Version minimum	Lien de telechargement
Node.js	18.x ou 20.x	https://nodejs.org
npm	9.x+ (inclus avec Node)	—
Git	2.x+	https://git-scm.com
Stripe CLI	Derniere version	<code>scoop install stripe</code> (Windows)
Navigateur moderne	Chrome/Edge/Firefox	—
Editeur de code	VS Code recommande	https://code.visualstudio.com

Comptes a creer (gratuits)

Service	URL	Role
Sanity.io	https://www.sanity.io	CMS (gestion produits, categories)

Supabase	https://supabase.com	Auth + Base de donnees
Stripe	https://dashboard.stripe.com	Paielements en ligne
Cal.com	https://cal.com	Prise de rendez-vous (optionnel)
Vercel	https://vercel.com	Hebergement (recommande)

Verifier l'installation

Ouvrir un terminal et executer :

```
node --version    # Doit afficher v18.x.x ou v20.x.x
npm --version     # Doit afficher 9.x.x+
git --version     # Doit afficher git version 2.x.x
```

2. Architecture du projet

```
my-shop/
├── apps/
│   ├── web/                                # Application Next.js (frontend + API)
│   │   ├── app/                            # Pages et routes (App Router)
│   │   │   ├── admin-hub/                 # Panel admin independant (login + dashboard)
│   │   │   ├── auth/                     # Pages d'authentification publiques
│   │   │   └── api/                       # Routes API (checkout, webhook)
│   │   ├── components/                   # Composants React reutilisables
│   │   │   └── admin/                     # Composants admin (AdminHeader, AdminHubCard)
│   │   ├── lib/                          # Clients (Sanity, Supabase, Stripe)
│   │   │   └── auth/                      # Guards (requireAdmin)
│   │   ├── hooks/                        # Hooks React personnalisés
│   │   ├── types/                        # Types TypeScript
│   │   ├── styles/                       # CSS global + config Tailwind
│   │   ├── public/                       # Fichiers statiques (images, favicon)
│   │   ├── .env.local                     # Variables d'environnement (SECRETS)
│   │   └── .env.example                   # Template des variables (sans secrets)
│   └── studio/                           # Sanity Studio (back-office CMS)
│       ├── schemaTypes/                   # Schemas : Product, Category, Order
│       ├── .env                           # Config Sanity (project ID, dataset)
│       └── sanity.config.ts                # Configuration du studio
├── supabase/
│   └── migrations/                        # Scripts SQL (tables, RLS, indexes)
├── seed/                                  # Script de peuplement Sanity
└── docker/                               # Configuration Docker (optionnel)
```

Roles de chaque service

Service	Gere quoi	Interface admin
Sanity Studio	Produits, categories, images, descriptions, tags, cross-sell	localhost:3333
Supabase	Comptes clients, paniers persistants, commandes, avis, fidelite	Dashboard web
Stripe Dashboard	Paiements, remboursements, factures, litiges	Dashboard web
Cal.com	Creneaux RDV, confirmations, annulations	Dashboard web

3. Installation locale

Etape 3.1 — Cloner le projet

```
git clone <URL_DU_REPO> my-shop
cd my-shop
```

Etape 3.2 — Installer les dependances

```
# Application web
cd apps/web
npm install

# Studio Sanity
cd ../studio
npm install
```

Etape 3.3 — Creer les fichiers d'environnement

```
# Application web
cd apps/web
cp .env.example .env.local

# Studio Sanity
cd ../studio
cp .env.example .env
```

SECURITE : Les fichiers `.env.local` et `.env` contiennent des secrets. Ils sont deja dans `.gitignore` — ne JAMAIS les

commiter dans Git. Ne JAMAIS partager ces fichiers par email, Slack ou tout autre canal non sécurisé.

Etape 3.4 — Verifier le .gitignore

Ouvrir `.gitignore` à la racine et confirmer la présence de :

```
.env
.env.local
.env.development.local
.env.test.local
.env.production.local
```

4. Configuration Sanity (CMS)

Sanity gère le catalogue de produits (noms, prix, images, catégories, descriptions). Sans Sanity configuré, l'application affiche des produits de démonstration (mock data).

Etape 4.1 — Créer un compte Sanity

1. Aller sur <https://www.sanity.io>
2. Cliquer "Get started" puis "Sign up"
3. Créer un compte (Google, GitHub, ou email)

Etape 4.2 — Créer un nouveau projet Sanity

1. Aller sur <https://www.sanity.io/manage>
2. Cliquer "Create new project"
3. Remplir :
 - **Project name** : Nom du client (ex: `Epicerie Africaine - Client XYZ`)
 - **Use the default dataset configuration** : Oui (cela crée un dataset `production`)
4. **Noter le Project ID** affiché en haut (ex: `abc12xyz`) — on en aura besoin

Etape 4.3 — Configurer les CORS Origins

1. Dans le dashboard Sanity (<https://www.sanity.io/manage>)
2. Sélectionner le projet
3. Aller dans **API > CORS Origins**
4. Ajouter ces origines :
 - `http://localhost:3007` (développement)
 - `http://localhost:3333` (studio local)
 - `https://votre-domaine.com` (production — à ajouter plus tard)
5. Pour chaque origine, cocher "Allow credentials"

Etape 4.4 — Créer un token API

1. Dans le dashboard Sanity > **API > Tokens**
2. Cliquer "Add API token"

3. Remplir :

- **Token name** : Next.js App
- **Permissions** : **Editor** (lecture + ecriture)

4. Cliquer "**Save**"

5. **COPIER LE TOKEN IMMEDIATEMENT** — il ne sera plus visible apres

ATTENTION : Ce token donne acces en ecriture a tout le contenu. Ne JAMAIS l'exposer cote client. Il est utilise uniquement cote serveur.

Etape 4.5 — Remplir les variables d'environnement

Fichier `apps/web/.env.local` :

```
NEXT_PUBLIC_SANITY_PROJECT_ID=votre_project_id_ici
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2024-01-01
SANITY_API_TOKEN=votre_token_api_ici
```

Fichier `apps/studio/.env` :

```
SANITY_STUDIO_PROJECT_ID=votre_project_id_ici
SANITY_STUDIO_DATASET=production
```

Etape 4.6 — Mettre a jour la config du studio

Ouvrir `apps/studio/sanity.config.ts` et verifier que le `projectId` correspond :

```
export default defineConfig({
  name: 'default',
  title: 'Epicerie Africaine',
  projectId: 'votre_project_id_ici', // <-- Mettre le bon ID
  dataset: 'production',
  // ...
})
```

Etape 4.7 — Lancer le studio et verifier

```
cd apps/studio
npm run dev
```

Le studio s'ouvre sur `http://localhost:3333` . Verifier que vous voyez les sections : **Product, Category, Order**.

Etape 4.8 — (Optionnel) Peupler avec des données de demo

Si un script de seed existe :

```
cd seed
node seed.js
```

Sinon, passer à la section 8 pour alimenter manuellement le catalogue.

5. Configuration Supabase (Auth + BDD)

Supabase gère l'authentification (inscription/connexion), les profils utilisateur, les paniers persistants, les commandes et les points de fidélité.

Sans Supabase, l'app fonctionne en **mode invite** (panier localStorage, pas de comptes).

Etape 5.1 — Créer un compte Supabase

1. Aller sur <https://supabase.com>
2. Cliquer "**Start your project**"
3. Se connecter avec GitHub (recommandé) ou email

Etape 5.2 — Créer un nouveau projet

1. Cliquer "**New Project**"
2. Remplir :
 - **Organization** : Sélectionner ou créer une organisation
 - **Name** : Nom du client (ex: `epicerie-africaine-clientxyz`)
 - **Database Password** : Générer un mot de passe fort et **LE NOTER QUELQUE PART** (coffre-fort de mots de passe recommandé)
 - **Region** : Choisir la plus proche du client
 - Canada : `East US (North Virginia)` ou `Central Canada`
 - France : `West EU (Ireland)` ou `Central EU (Frankfurt)`
3. Cliquer "**Create new project**"
4. **Attendre ~2 minutes** que le projet soit provisionné

Etape 5.3 — Récupérer les clés API

1. Dans le dashboard Supabase, aller dans **Settings** (icône engrenage) > **API**
2. Vous verrez :

Project URL	
<code>https://abcdefghijkl.supabase.co</code>	[Copy]
Project API keys	
<code>anon public <votre_cle_jwt></code>	[Copy]

```
| service_role  <votre_cle_jwt>  [Copy]  |
```

3. Copier chaque valeur

Etape 5.4 — Remplir les variables d'environnement

Fichier `apps/web/.env.local` :

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://abcdefghijkl.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<votre_cle_jwt>
SUPABASE_SERVICE_ROLE_KEY=<votre_cle_jwt>
```

SECURITE CRITIQUE : La cle `service_role` bypass TOUTES les regles de securite (RLS). Elle ne doit JAMAIS etre exposee cote client (jamais dans une variable `NEXT_PUBLIC_`). Elle est utilisee uniquement dans les routes API serveur (`/api/webhook/stripe`).

Etape 5.5 — Creer les tables (migrations SQL)

1. Dans le dashboard Supabase, aller dans **SQL Editor** (icone terminal dans la sidebar)
2. Cliquer **"New Query"**
3. Ouvrir le fichier `supabase/migrations/` de votre projet
4. **Option rapide** : Copier-coller TOUT le contenu de `supabase/migrations/all.sql` et cliquer **"Run"**

Option detaillee (fichier par fichier, dans l'ordre) :

- `001_init.sql` — Tables principales + trigger auto-profil
- `002_rls_policies.sql` — Regles de securite Row Level Security
- `003_indexes.sql` — Index de performance
- `004_loyalty_rpc.sql` — Fonction points de fidelite
- `005_admin_role.sql` — Colonne role (admin/customer) dans profiles
- `006_security_fixes.sql` — Correctifs securite (protection role, RPC, search_path)
- `007_audit_logs.sql` — Table audit_logs pour tracabilite admin

5. Pour chaque fichier, coller le SQL et cliquer **"Run"** (ou `Ctrl+Enter`)
6. Verifier le message : **"Success. No rows returned."**

Etape 5.6 — Verifier les tables creees

Aller dans **Table Editor** (sidebar) et confirmer la presence de :

Table	Description	Colonnes principales
<code>profiles</code>	Profils utilisateur	id, full_name, phone, loyalty_points, role (admin/customer)

cartes	Paniers persistants	id, user_id, status (active/converted)
cart_items	Articles dans les paniers	cart_id, product_slug, name, price_cents, quantity
orders	Commandes (creees par webhook)	stripe_session_id, status, customer_email, total_cents
order_items	Articles dans les commandes	order_id, product_slug, name, price_cents, quantity
reviews	Avis clients	user_id, product_slug, rating, comment
audit_logs	Tracabilite admin	actor_id, actor_email, action, resource, metadata

Etape 5.7 — Configurer l'authentification

1. Aller dans **Authentication** (icone cadenas) > **URL Configuration**
2. Configurer :
 - **Site URL** : `http://localhost:3007` (dev) ou `https://votre-domaine.com` (prod)
 - **Redirect URLs** : Cliquer "Add URL" et ajouter :
 - `http://localhost:3007/auth/callback`
 - `https://votre-domaine.com/auth/callback` (ajouter pour la prod)

Etape 5.8 — (Dev uniquement) Desactiver la confirmation email

Pour faciliter les tests en developpement :

1. **Authentication > Providers > Email**
2. Decocher "**Confirm email**"
3. Sauvegarder

IMPORTANT : Reactiver cette option avant la mise en production !

Etape 5.9 — Verifier les politiques RLS

1. Aller dans **Authentication > Policies**
2. Confirmer que chaque table a ses politiques de securite :
 - `profiles` : lecture/modification de son propre profil (le champ `role` ne peut PAS etre modifie par l'utilisateur)
 - `cartes` / `cart_items` : CRUD sur ses propres paniers
 - `orders` / `order_items` : lecture de ses propres commandes
 - `reviews` : lecture publique, CRUD sur ses propres avis

6. Configuration Stripe (Paielements)

Stripe gere le paiement en ligne via Checkout Sessions. Sans Stripe, le bouton "Payer" ne fonctionnera pas.

Etape 6.1 — Creer un compte Stripe

1. Aller sur <https://dashboard.stripe.com/register>
2. Creer un compte (email + mot de passe)
3. Pas besoin de verifier son identite pour le **mode test**

Etape 6.2 — Activer le mode test

1. Dans le dashboard Stripe, vérifier en haut à droite
2. Le toggle "**Test mode**" doit être **active** (couleur orange)
3. Si ce n'est pas le cas, cliquer dessus pour l'activer

En mode test, aucun vrai paiement n'est effectué. Les cartes de test simulent les transactions.

Etape 6.3 — Récupérer les clés API

1. Aller dans **Developers > API keys**
2. Ou directement : <https://dashboard.stripe.com/test/apikeys>
3. Vous verrez :

Publishable key	pk_test_XXXX...	[Copy]
Secret key	sk_test_XXXX...	[Reveal] [Copy]

4. Copier les deux clés

Etape 6.4 — Remplir les variables d'environnement

Fichier `apps/web/.env.local` :

```
STRIPE_SECRET_KEY=<votre_cle_secrete_test>
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=<votre_cle_publique_test>
STRIPE_WEBHOOK_SECRET=<votre_webhook_secret> # Rempli à l'étape suivante
```

SECURITE : La clé secrète (`sk_test_...` ou `sk_live_...`) ne doit JAMAIS être dans une variable `NEXT_PUBLIC_`.

Etape 6.5 — Configurer le webhook (développement local)

Le webhook est essentiel : il reçoit la confirmation de paiement de Stripe et crée la commande dans la base de données.

A. Installer Stripe CLI

```
# Windows (avec Scoop)
scoop install stripe

# Windows (avec Chocolatey)
choco install stripe-cli

# macOS
brew install stripe/stripe-cli/stripe
```

```
# Linux (Debian/Ubuntu)
curl -s https://packages.stripe.dev/api/security/keypair/stripe-cli-gpg/public | gpg --dearmor | sudo tee /usr/share/keyrings/stripe.gpg
echo "deb [signed-by=/usr/share/keyrings/stripe.gpg] https://packages.stripe.dev/stripe-cli-debian-local stable main" | sudo tee -a /etc/apt/sources.list.d/stripe.list
sudo apt update && sudo apt install stripe
```

B. Se connecter a Stripe

```
stripe login
```

Un navigateur s'ouvre. Cliquer **"Allow access"** pour autoriser la CLI.

C. Ecouter les webhooks en local

Dans un **terminal separé** (laisser tourner pendant le developpement) :

```
stripe listen --forward-to localhost:3007/api/webhook/stripe
```

La commande affiche :

```
Ready! Your webhook signing secret is <votre_webhook_secret>
```

D. Copier le secret webhook

Copier le `whsec_...` et le coller dans `apps/web/.env.local` :

```
STRIPE_WEBHOOK_SECRET=<votre_webhook_secret>
```

NOTE : Ce secret change a chaque fois que vous relancez `stripe listen`. En production, il est fixe (voir section 10).

Etape 6.6 — Cartes de test Stripe

Scenario	Numero de carte	Date	CVC
Paiement reussi	4242 4242 4242 4242	12/34	123
Paiement refuse	4000 0000 0000 0002	12/34	123
Authentification 3D Secure	4000 0025 0000 3155	12/34	123
Fonds insuffisants	4000 0000 0000 9995	12/34	123

Etape 6.7 — Configurer le webhook (production)

Quand le site est déployé en production :

1. Aller dans **Developers > Webhooks** dans le dashboard Stripe
2. Cliquer "**Add endpoint**"
3. Remplir :
 - **Endpoint URL** : `https://votre-domaine.com/api/webhook/stripe`
 - **Events to send** : Sélectionner `checkout.session.completed`
4. Cliquer "**Add endpoint**"
5. Copier le "**Signing secret**" affiché
6. Le mettre dans la variable `STRIPE_WEBHOOK_SECRET` de l'hébergeur (Vercel)

7. Configuration Cal.com (Rendez-vous)

Cal.com permet aux clients de prendre rendez-vous en ligne. **Ce service est optionnel.** Sans configuration, la page `/appointments` affiche un fallback avec un lien de contact.

Etape 7.1 — Créer un compte Cal.com

1. Aller sur <https://cal.com>
2. Cliquer "**Get Started**"
3. Créer un compte (Google, email, etc.)
4. Choisir un username (ex: `epicerie-africaine`)

Etape 7.2 — Créer un Event Type

1. Dans le dashboard Cal.com, aller dans **Event Types**
2. Cliquer "**New Event Type**"
3. Configurer :
 - **Title** : ex. "Consultation en boutique"
 - **Duration** : 30 minutes (ou selon le besoin)
 - **Location** : En personne / Telephone / Video
4. Configurer les **disponibilités** (jours, heures)
5. Sauvegarder

Etape 7.3 — Récupérer l'URL de l'événement

1. Dans **Event Types**, cliquer sur l'événement créé
2. L'URL est en haut, au format : `votre-username/nom-event`
3. Exemple : `epicerie-africaine/consultation`

Etape 7.4 — Remplir la variable d'environnement

Fichier `apps/web/.env.local` :

```
NEXT_PUBLIC_CALCOM_EMBED_URL=epicerie-africaine/consultation
```

NOTE : Mettre uniquement `username/event-name` , pas l'URL complete.

8. Alimenter le catalogue (Sanity Studio)

Etape 8.1 — Lancer le studio

```
cd apps/studio
npm run dev
```

Ouvrir <http://localhost:3333> dans le navigateur.

Etape 8.2 — Créer les categories

Aller dans **Category** > cliquer le bouton "+" pour creer.

Categories recommandees pour une epicerie africaine :

#	Titre	Slug (auto)	Ordre	Description
1	Epices	epices	1	Epices et condiments d'Afrique
2	Cereales et Feculents	cereales-et-feculents	2	Riz, semoule, farine, igname...
3	Sauces et Pates	sauces-et-pates	3	Sauces tomate, pate d'arachide...
4	Boissons	boissons	4	Jus, bissap, gingembre...
5	Snacks	snacks	5	Chips de plantain, chin-chin...
6	Huiles	huiles	6	Huile de palme, huile de coco...
7	Produits frais	produits-frais	7	Plantain, manioc, gombo...
8	Soins et Beaute	soins-et-beaute	8	Beurre de karite, savon noir...

Pour chaque categorie :

1. Remplir le **titre**
2. Cliquer "**Generate**" a cote du slug pour le generer automatiquement
3. Mettre le numero d'**ordre** (pour le tri)
4. Ajouter une **description** courte
5. Uploader une **image** representative (format carre, min 600x600px)
6. Cliquer "**Publish**"

Etape 8.3 — Créer les produits

Aller dans **Product** > cliquer "+" pour creer un nouveau produit.

Pour chaque produit, remplir **TOUS** les champs suivants :

Champs obligatoires :

Champ	Description	Exemple
Title	Nom du produit	"Piment Camerounais Fume"
Slug	Cliquer "Generate"	piment-camerounais-fume
Price	Prix en dollars (nombre)	8.99
Currency	Devise	CAD (par défaut)
Images	Min 1 image, idéalement 3-4	Photos HD du produit
Category	Reference a une categorie	Sélectionner "Epices"
Stock	Nombre d'unités disponibles	50

IMPORTANT : Si le stock est à 0, le bouton "Ajouter au panier" sera remplacé par "Rupture de stock" et le client ne pourra pas acheter.

Champs recommandés :

Champ	Description	Exemple
Description	Texte riche (gras, italique, listes)	Description détaillée du produit
Tags	Étiquettes filtrable	Bio, Piment, Surgelé
Origin Country	Pays d'origine	Cameroun
Spicy Level	0-3	2 (Moyen)
Is Frozen	Produit surgelé ?	Non
Is Organic	Produit bio ?	Oui
Is Featured	Afficher en best-seller sur l'accueil	Oui (8 max recommandés)
Related Products	Produits similaires/complémentaires	Sélectionner 2-4 produits

Conseils pour les images :

- **Format** : JPEG ou WebP (plus léger)
- **Taille** : Minimum 800x800px, idéalement 1200x1200px
- **Ratio** : Carré (1:1) pour uniforme dans la grille
- **Fond** : Fond blanc ou neutre pour un rendu professionnel
- **Alt text** : Toujours remplir le texte alternatif (SEO + accessibilité)
 - Exemple : "Piment camerounais fume dans un bol en bois"

Étape 8.4 — Nombre minimum de produits recommandés

Type de boutique	Nombre minimum	Idéal
------------------	----------------	-------

Demo / MVP	10 produits	15
Boutique standard	20 produits	30-50
Grande boutique	50+ produits	100+

Etape 8.5 — Verifier la publication

Apres avoir cree les produits :

1. S'assurer que **CHAQUE produit et categorie est publie** (bouton "Publish")
2. Aller sur `http://localhost:3007/shop` et rafraichir la page
3. Les produits doivent apparaitre dans la grille
4. Tester les filtres (categorie, tags, prix, recherche)

9. Configuration Admin Hub

Le panel admin (`/admin-hub`) est une zone completement independante du site public. Il possede son propre formulaire de connexion, son propre header et son propre design (dark mode).

Etape 9.1 — Comprendre l'architecture admin

Element	Description
<code>/admin-hub/login</code>	Formulaire de connexion admin (dark mode, pas de creation de compte)
<code>/admin-hub</code>	Dashboard avec 4 cartes vers les services externes
<code>AdminHeader</code>	Header minimaliste : badge Admin, Voir le site, Deconnexion
<code>requireAdmin()</code>	Guard serveur qui verifie le role <code>admin</code> dans la table <code>profiles</code>

Differences avec le site public :

- Pas de header/footer du site public
- Pas de possibilite de creer un compte
- Design dark mode independant
- Verrouillage apres 5 tentatives de connexion echouees
- Pages non indexees par les moteurs de recherche

Etape 9.2 — Creer le premier administrateur

IMPORTANT : Le role `admin` ne peut etre attribue que via SQL. Aucun utilisateur ne peut se promouvoir lui-meme grace a la protection RLS.

Option A — Via le dashboard Supabase (recommande)

1. Aller dans **Authentication > Users > Add user > Create new user**
2. Remplir :
 - **Email** : l'email de l'administrateur

- **Password** : un mot de passe fort (minimum 8 caractères)
- Cocher **Auto Confirm User**

3. Cliquer **Create user**

4. Aller dans **SQL Editor > New Query**

5. Exécuter cette requête (remplacer l'email) :

```
UPDATE profiles SET role = 'admin'
WHERE id = (SELECT id FROM auth.users WHERE email = 'admin@exemple.com');
```

6. Vérifier avec :

```
SELECT p.role, u.email
FROM profiles p
JOIN auth.users u ON u.id = p.id
WHERE p.role = 'admin';
```

Option B — Via la ligne de commande (Supabase CLI)

```
# 1. Installer le CLI
npm install -g supabase

# 2. Se connecter
supabase login

# 3. Lier le projet (le project-ref est dans l'URL du dashboard)
supabase link --project-ref VOTRE_PROJECT_REF

# 4. Promouvoir en admin
supabase db execute --sql "UPDATE profiles SET role = 'admin' WHERE id = (SELECT id FROM auth.users
WHERE email = 'admin@exemple.com');"
```

Etape 9.3 — Configurer les URLs des dashboards (optionnel)

Dans `apps/web/.env.local`, vous pouvez personnaliser les liens des cartes admin :

```
# URLs des dashboards (optionnel – des fallbacks sont utilisés par défaut)
NEXT_PUBLIC_SANITY_STUDIO_URL=/studio
NEXT_PUBLIC_STRIPE_DASHBOARD_URL=https://dashboard.stripe.com/
NEXT_PUBLIC_SUPABASE_DASHBOARD_URL=https://supabase.com/dashboard
NEXT_PUBLIC_CAL_DASHBOARD_URL=https://app.cal.com/availability
```

Etape 9.4 — Tester l'accès admin

1. Lancer le serveur : `npm run dev -- -p 3007`

2. Aller sur `http://localhost:3007/admin-hub`
3. Vous devez etre redirige vers `/admin-hub/login`
4. Se connecter avec le compte admin cree a l'etape 9.2
5. Verifier :
 - Le dashboard s'affiche avec les 4 cartes
 - Le header affiche : badge Admin, Voir le site, Deconnexion
 - "Voir le site" ouvre le site dans un nouvel onglet
 - "Deconnexion" redirige vers `/admin-hub/login`
 - Un compte non-admin est refuse avec le message "Acces refuse"

10. Securite

10.1 — Headers HTTP de securite

Tous les headers sont configures dans `apps/web/next.config.mjs` :

Header	Protection
<code>Content-Security-Policy</code>	Empeche l'injection de scripts malveillants (XSS)
<code>Strict-Transport-Security</code>	Force HTTPS avec preload (2 ans)
<code>X-Frame-Options: DENY</code>	Empeche l'integration du site dans une iframe (clickjacking)
<code>X-Content-Type-Options: nosniff</code>	Empeche le navigateur de deviner le type MIME
<code>Referrer-Policy</code>	Controle les informations envoyees dans le header Referer
<code>Permissions-Policy</code>	Desactive camera, micro, geoloc, paiement, USB, gyroscope, capture ecran
<code>Cross-Origin-Opener-Policy</code>	Isole la fenetre des attaques cross-origin
<code>Cross-Origin-Resource-Policy</code>	Protege contre les attaques side-channel (Spectre)
<code>Cache-Control</code> (API)	<code>no-store, max-age=0</code> sur toutes les routes <code>/api/*</code>

Le header `X-Powered-By` est supprime (`poweredByHeader: false`) pour masquer le framework.

10.2 — Protections de la base de donnees

Protection	Description
RLS (Row Level Security)	Active sur TOUTES les tables. Chaque utilisateur ne voit que ses propres donnees
Protection du role	Un utilisateur ne peut PAS modifier son propre champ <code>role</code> (politique <code>WITH CHECK</code>)
Fonction loyalty restreinte	<code>increment_loyalty</code> ne peut etre appelee que par le <code>service_role</code> (webhook)
search_path securise	Toutes les fonctions <code>SECURITY DEFINER</code> ont <code>SET search_path = public</code>

10.3 — Protections applicatives

Protection	Description
Vérification des prix	Les prix sont recuperes depuis Sanity cote serveur au checkout (jamais les prix du client)
Signature webhook	Chaque requete webhook Stripe est verifiee avec <code>constructEvent()</code>
Idempotence	Les commandes dupliquees sont detectees via <code>stripe_session_id</code> UNIQUE + <code>maybeSingle()</code>
Open redirect	Toutes les redirections auth sont validees (doit commencer par <code>/</code> , pas <code>//</code>)
Mot de passe	Minimum 8 caracteres (norme NIST SP 800-63B)
Cookies	<code>SameSite=Lax</code> + <code>Secure=true</code> en production
Erreurs masquées	Les erreurs webhook ne revelent pas de details internes
CVE-2025-29927	Header <code>x-middleware-subrequest</code> bloque pour empecher le bypass du middleware
Honeypot anti-bot	Champ invisible sur le checkout rejette les soumissions automatisees
Session admin	Expiration forcee apres 4 heures, re-authentification obligatoire
Audit logs	Table <code>audit_logs</code> pour tracer toutes les actions admin sensibles
Service client	Validation obligatoire des variables <code>SUPABASE_SERVICE_ROLE_KEY</code> avant utilisation
security.txt	Route <code>/.well-known/security.txt</code> conforme RFC 9116

10.4 — Regles de securite a respecter

CRITIQUE : Ne JAMAIS faire les actions suivantes :

1. Prefixer une cle secrete avec `NEXT_PUBLIC_` (elle serait exposee dans le navigateur)
2. Commiter un fichier `.env.local` ou `.env` dans Git
3. Partager la cle `SUPABASE_SERVICE_ROLE_KEY` par email ou Slack
4. Desactiver RLS sur une table en production
5. Utiliser `getSession()` au lieu de `getUser()` pour verifier l'authentification serveur
6. Mettre des mots de passe dans `.env.example` ou dans le code source

10.5 — Migration de securite (bases existantes)

Si le projet a deja ete deploye AVANT ces correctifs, executer ces migrations :

1. Aller dans **Supabase > SQL Editor > New Query**
2. Copier-coller le contenu de `supabase/migrations/006_security_fixes.sql` → **Run**
3. Copier-coller le contenu de `supabase/migrations/007_audit_logs.sql` → **Run**
4. Verifier : **"Success. No rows returned."**

Migration 006 corrige :

- La politique RLS `profiles` (empeche la modification du role)
- L'accès public a `increment_loyalty` (revoque)

- Le `search_path` sur les fonctions `SECURITY DEFINER`

Migration 007 ajoute :

- Table `audit_logs` pour tracer les actions admin (connexion, modifications, etc.)
- RLS total : aucun accès via l'API publique, écriture uniquement par `service_role`

11. Tests fonctionnels

Avant de livrer le site, effectuer TOUS les tests suivants.

Test 1 : Navigation generale

- ☐ Page d'accueil (`/`) : hero, categories, best-sellers s'affichent
- ☐ Page boutique (`/shop`) : tous les produits avec filtres
- ☐ Page produit (`/product/[slug]`) : image, prix, description, produits similaires
- ☐ Pages legales (`/legal/terms` , `/legal/privacy` , `/legal/refunds` , `/legal/imprint`)
- ☐ Page rendez-vous (`/appointments`) : embed Cal.com ou fallback
- ☐ Header : logo, navigation, icone panier avec badge
- ☐ Footer : liens fonctionnels vers toutes les sections
- ☐ Responsive : tester sur mobile (375px), tablette (768px), desktop (1440px)

Test 2 : Panier

- ☐ Ajouter un produit depuis la page boutique (bouton "+")
- ☐ Ajouter un produit depuis la page produit detail
- ☐ Le badge du panier se met a jour en temps reel
- ☐ Page panier (`/cart`) : produits affiches avec images, prix, quantites
- ☐ Modifier la quantite (+ / -)
- ☐ Supprimer un article
- ☐ Le total se recalcule correctement
- ☐ Livraison gratuite affichee si total $\geq$ 75\$
- ☐ Panier persiste apres rafraichissement de la page (localStorage)

Test 3 : Checkout et paiement

Pre-requis : Stripe et Supabase configurees, `stripe listen` en cours.

- ☐ Ajouter des produits au panier puis aller sur `/checkout`
- ☐ **Mode livraison** :
 - ☐ Formulaire : nom, email, telephone, adresse complete
 - ☐ Frais de livraison affiches (5.99\$ ou gratuit si $\geq$ 75\$)
 - ☐ Cliquer "Payer" → redirection vers Stripe Checkout
 - ☐ Payer avec `4242 4242 4242 4242 / 12/34 / 123`
 - ☐ Redirection vers `/success` avec details de la commande
 - ☐ Panier vide apres le paiement
- ☐ **Mode retrait (pickup)** :

- ☐ Sélectionner un créneau de retrait
- ☐ Pas de frais de livraison
- ☐ Meme flux de paiement
- ☐ Vérifier dans le **terminal Stripe CLI** : événement `checkout.session.completed` reçu
- ☐ Vérifier dans **Supabase > Table Editor > orders** : commande créée
- ☐ Vérifier dans **Stripe Dashboard > Payments** : paiement visible

Test 4 : Authentification

Pre-requis : Supabase configuré.

- ☐ **Inscription** (`/auth/register`) : créer un compte avec email/mot de passe
- ☐ Si confirmation email désactivée : connexion immédiate
- ☐ Si confirmation email activée : vérifier l'email puis se connecter
- ☐ **Connexion** (`/auth/login`) : se connecter avec le compte créé
- ☐ Le header affiche l'icône utilisateur au lieu de "Se connecter"
- ☐ **Page compte** (`/account`) : profil, historique de commandes, points de fidélité
- ☐ **Déconnexion** : fonctionne correctement, retour en mode invité
- ☐ **Fusion panier** : ajouter des produits en invité → se connecter → les produits sont conservés

Test 5 : Admin Hub

Pre-requis : Supabase configuré, utilisateur admin créé (section 9).

- ☐ `/admin-hub` redirige vers `/admin-hub/login` si non connecté
- ☐ Formulaire admin : design dark mode, pas de lien "Créer un compte"
- ☐ Connexion avec un compte admin : accès au dashboard
- ☐ Connexion avec un compte client (non admin) : message "Accès refusé"
- ☐ Verrouillage après 5 tentatives échouées
- ☐ Dashboard : 4 cartes visibles (Sanity, Stripe, Supabase, Cal.com)
- ☐ Header admin : badge Admin, "Voir le site" (nouvel onglet), "Déconnexion"
- ☐ Déconnexion redirige vers `/admin-hub/login`
- ☐ Pas de header/footer du site public visible sur les pages admin
- ☐ Les pages admin ne sont pas indexées (vérifier avec `robots.txt` ou `<meta>`)

Test 6 : Sécurité

- ☐ Headers HTTP présents : vérifier avec <https://securityheaders.com>
- ☐ X-Powered-By absent des réponses HTTP
- ☐ Un utilisateur normal ne peut PAS modifier son rôle dans `profiles`
 - Tester via Supabase JS : `supabase.from('profiles').update({role:'admin'}).eq('id', user.id)` → doit échouer
- ☐ `increment_loyalty` non appelable par un utilisateur authentifié
 - Tester : `supabase.rpc('increment_loyalty', {user_id_input: id, points_input: 9999})` → doit échouer
- ☐ Redirect auth : impossible de rediriger vers un site externe après login

Test 7 : Cas limites

- ☐ Produit en rupture de stock (stock = 0) : bouton "Rupture de stock" désactive
- ☐ Checkout avec panier vide : redirection vers la boutique
- ☐ Paiement refusé (carte 4000 0000 0000 0002) : message d'erreur correct
- ☐ Page 404 : tester une URL inexistante (ex: /page-qui-n'existe-pas)

12. Déploiement en production

Option recommandée : Vercel

Étape 12.1 — Créer un compte Vercel

1. Aller sur <https://vercel.com>
2. Se connecter avec **GitHub** (recommandé)
3. Autoriser l'accès au repository du projet

Étape 12.2 — Importer le projet

1. Cliquer "**Add New**" > "**Project**"
2. Sélectionner le repository Git
3. Configurer :
 - **Framework Preset** : Next.js (auto-détecte)
 - **Root Directory** : apps/web
 - **Build Command** : npm run build (par défaut)
 - **Output Directory** : .next (par défaut)

Étape 12.3 — Configurer les variables d'environnement

Dans **Settings > Environment Variables**, ajouter TOUTES les variables :

```
# Sanity
NEXT_PUBLIC_SANITY_PROJECT_ID=votre_project_id
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2024-01-01
SANITY_API_TOKEN=votre_token

# Supabase
NEXT_PUBLIC_SUPABASE_URL=<votre_url_supabase>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<votre_anon_key>
SUPABASE_SERVICE_ROLE_KEY=<votre_service_role_key>

# Stripe (MODE LIVE !)
STRIPE_SECRET_KEY=<votre_cle_secrete_stripe>
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=<votre_cle_publique_stripe>
STRIPE_WEBHOOK_SECRET=<votre_webhook_secret_stripe>

# Cal.com
NEXT_PUBLIC_CALCOM_EMBED_URL=username/event

# App
NEXT_PUBLIC_BASE_URL=https://votre-domaine.com
```

```
NEXT_PUBLIC_SITE_NAME=Epicerie Africaine
```

```
# Shipping
```

```
NEXT_PUBLIC_SHIPPING_COST=5.99
```

```
NEXT_PUBLIC_FREE_SHIPPING_THRESHOLD=75
```

SECURITE : En production, utiliser les clés **LIVE** de Stripe (`sk_Live_` , `pk_Live_`), pas les clés de test. Vérifier que l'identité est validée dans le dashboard Stripe avant de passer en live.

Etape 12.4 — Deployer

1. Cliquer "**Deploy**"
2. Attendre la fin du build (~2-3 minutes)
3. Vercel fournit une URL (ex: `epicerie-africaine.vercel.app`)

Etape 12.5 — Configurer le domaine personnalisé

1. Dans Vercel, aller dans **Settings > Domains**
2. Ajouter le domaine du client (ex: `epicerie-africaine.ca`)
3. Suivre les instructions pour configurer les DNS :
 - **Type A** : `76.76.21.21`
 - **Type CNAME** (www) : `cname.vercel-dns.com`
4. Attendre la propagation DNS (~5-30 minutes)
5. Vercel configure automatiquement le certificat SSL (HTTPS)

Etape 12.6 — Mises à jour post-déploiement

Après le déploiement, mettre à jour :

1. `NEXT_PUBLIC_BASE_URL` dans Vercel : `https://votre-domaine.com`
2. **Supabase** > Authentication > URL Configuration :
 - Site URL : `https://votre-domaine.com`
 - Redirect URLs : ajouter `https://votre-domaine.com/auth/callback`
3. **Sanity** > API > CORS Origins : ajouter `https://votre-domaine.com`
4. **Stripe** > Webhooks : créer l'endpoint de production (voir section 6.7)
5. **Supabase** > Reactiver la confirmation email si désactivée

13. Personnalisation pour un nouveau client

Quand tu configures le site pour un nouveau client, voici les éléments à adapter :

13.1 — Branding

Element	Fichier(s) à modifier	Quoi changer
Nom du site	<code>.env.local</code> (<code>NEXT_PUBLIC_SITE_NAME</code>)	Nom du client
Logo	<code>apps/web/components/Header.tsx</code> et <code>Footer.tsx</code>	Remplacer le logo/texte
Couleurs	<code>apps/web/tailwind.config.ts</code>	Modifier les couleurs accent

Favicon	apps/web/public/favicon.ico	Remplacer
Meta SEO	apps/web/app/layout.tsx	Title, description, OG images

13.2 — Couleurs (Design Tokens)

Les couleurs sont centralisées dans `tailwind.config.ts` :

```
colors: {
  background: '#F9F9F7', // Fond principal
  foreground: '#1A1A1A', // Texte principal
  accent: {
    DEFAULT: '#CCA43B', // Or / doré (CTAs, liens)
    light: '#E0C36A', // Version claire
    dark: '#B08A2A', // Version foncée
  }
}
```

13.3 — Contenu legal

Page	Fichier	A adapter
CGV	apps/web/app/legal/terms/page.tsx	Nom société, domaine, juridiction
Confidentialité	apps/web/app/legal/privacy/page.tsx	Responsable, DPO, cookies
Retours	apps/web/app/legal/refunds/page.tsx	Délais, conditions
Mentions légales	apps/web/app/legal/imprint/page.tsx	Raison sociale, SIRET/NEQ, adresse

13.4 — Livraison

Dans `.env.local` :

```
NEXT_PUBLIC_SHIPPING_COST=5.99      # Frais de livraison
NEXT_PUBLIC_FREE_SHIPPING_THRESHOLD=75 # Seuil livraison gratuite
```

13.5 — Devise

La devise par défaut est **CAD** (Dollar canadien). Pour changer en EUR, modifier dans Sanity les produits (champ `currency`). Le format d'affichage est géré dans le composant `ProductCard`.

14. Maintenance et opérations courantes

Ajouter/modifier des produits

1. Ouvrir Sanity Studio (`npm run dev` dans `apps/studio/`)
2. Créer ou modifier les produits
3. Cliquer "**Publish**"
4. Les changements apparaissent sur le site en **~60 secondes** (revalidation ISR)

Gérer les commandes

1. Ouvrir le **dashboard Supabase** > Table Editor > `orders`
2. Les commandes sont créées automatiquement par le webhook Stripe
3. Statuts possibles : `pending` → `paid` → `shipped` → `delivered`
4. Pour mettre à jour un statut, modifier directement dans Supabase

Gérer les paiements

1. Ouvrir le **Stripe Dashboard**
2. Voir les paiements dans **Payments**
3. Effectuer des remboursements si nécessaire
4. Gérer les litiges dans **Disputes**

Voir les clients

1. **Supabase** > Authentication > Users : liste des comptes
2. **Supabase** > Table Editor > `profiles` : détails des profils

GitHub Action — Anti-pause Supabase (free tier)

Le projet Supabase free tier se met en pause automatiquement après 7 jours d'inactivité. Un GitHub Action (`.github/workflows/keep-alive.yml`) envoie un ping toutes les 72h pour empêcher cette pause.

Configuration des secrets GitHub

1. Aller dans **GitHub** > **Settings** > **Secrets and variables** > **Actions**
2. Cliquer "**New repository secret**" et ajouter :

Secret	Valeur
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL du projet Supabase (ex: <code>https://abcdef.supabase.co</code>)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Cle anonyme Supabase
<code>SITE_URL</code>	URL de production du site (ex: <code>https://votre-domaine.com</code>)

3. Le workflow s'exécute automatiquement tous les 3 jours. Vérifier dans **Actions** > **Keep Supabase Alive**.

NOTE : Si le projet passe en plan payant, ce workflow n'est plus nécessaire.

15. Checklist de livraison

Avant de livrer le site au client, verifier chaque point :

Configuration

- ☐ Sanity : projet cree, project ID configure, token API genere
- ☐ Supabase : projet cree, tables creees, RLS actif, auth configuree
- ☐ Stripe : compte cree, clef API configurees, webhook configure
- ☐ Cal.com : (si applicable) event type cree, URL configuree
- ☐ Variables d'environnement : TOUTES remplies dans `.env.local` ET sur l'hebergeur
- ☐ `.env.local` est dans `.gitignore`

Admin

- ☐ Utilisateur admin cree dans Supabase (section 9.2)
- ☐ Connexion admin testee sur `/admin-hub/login`
- ☐ Les 4 cartes du dashboard pointent vers les bons dashboards
- ☐ Migration 006 (securite) appliquee sur la base

Securite

- ☐ Migration `006_security_fixes.sql` executee
- ☐ Migration `007_audit_logs.sql` executee
- ☐ Headers HTTP verifies (CSP, HSTS, X-Frame-Options, CORP, Cache-Control API)
- ☐ Pas de mot de passe dans `.env.example` ou le code source
- ☐ `.env.local` dans `.gitignore`
- ☐ Clef secrete sans prefix `NEXT_PUBLIC_`
- ☐ `/.well-known/security.txt` accessible
- ☐ `SECURITY_CONTACT_EMAIL` configure dans les variables d'environnement

Contenu

- ☐ Categories creees dans Sanity (min 5)
- ☐ Produits creees dans Sanity (min 15-20) avec images HD
- ☐ Best-sellers marques (`isFeatured = true` , 8 max)
- ☐ Stock > 0 pour les produits disponibles
- ☐ Pages legales adaptees au client (nom, adresse, juridiction)

Tests

- ☐ Navigation : toutes les pages accessibles sans erreur
- ☐ Panier : ajout, modification, suppression, persistance
- ☐ Paiement : flux complet teste avec carte test
- ☐ Auth : inscription, connexion, deconnexion, page compte
- ☐ Responsive : teste sur mobile, tablette, desktop
- ☐ SEO : meta tags, robots.txt, sitemap presents

Production

- ☐ Deploye sur Vercel (ou equivalent)
- ☐ Domaine personnalise configure avec HTTPS
- ☐ Stripe en **mode Live** avec cles de production
- ☐ Webhook Stripe de production configure
- ☐ Confirmation email activee dans Supabase
- ☐ CORS Sanity mis a jour avec le domaine de prod
- ☐ URLs Supabase mises a jour avec le domaine de prod
- ☐ `NEXT_PUBLIC_BASE_URL` mis a jour avec le domaine de prod
- ☐ Secrets GitHub configures pour le workflow keep-alive (section 14)

Documentation client

- ☐ Acces Sanity Studio donne au client (pour gerer ses produits)
 - ☐ Formation basique sur l'ajout/modification de produits
 - ☐ Acces Stripe Dashboard (pour voir les paiements)
 - ☐ Procedure de remboursement expliquee
-

16. Depannage (FAQ)

"Les produits ne s'affichent pas"

1. Verifier que les produits sont **publies** dans Sanity Studio
2. Verifier que `NEXT_PUBLIC_SANITY_PROJECT_ID` est correct dans `.env.local`
3. Verifier les CORS dans Sanity (l'origine du site doit etre autorisee)
4. Attendre 60 secondes (revalidation ISR) et rafraichir

"Le paiement echoue"

1. Verifier que `STRIPE_SECRET_KEY` est correct
2. Verifier que `stripe listen` tourne (dev) ou que le webhook est configure (prod)
3. Verifier la console du navigateur et les logs serveur pour les erreurs
4. Tester avec la carte `4242 4242 4242 4242`

"L'inscription/connexion ne fonctionne pas"

1. Verifier que `NEXT_PUBLIC_SUPABASE_URL` et `NEXT_PUBLIC_SUPABASE_ANON_KEY` sont corrects
2. Verifier les Redirect URLs dans Supabase Authentication
3. Si "confirmation email" est activee : verifier la boite mail (et les spams)

"Le webhook ne cree pas de commande"

1. Verifier que `STRIPE_WEBHOOK_SECRET` correspond au secret affiche par `stripe listen`
2. Verifier que les tables `orders` et `order_items` existent dans Supabase
3. Verifier que `SUPABASE_SERVICE_ROLE_KEY` est correct
4. Regarder les logs du terminal ou le webhook echoue

"Erreur CORS"

1. Aller dans Sanity > API > CORS Origins
2. Ajouter l'URL exacte du site (avec `https://`)
3. Cocher "Allow credentials"
4. Sauvegarder et recharger la page

"Je ne peux pas me connecter a l'admin"

1. Verifier que le compte existe dans Supabase > Authentication > Users
2. Verifier que le role est bien `admin` dans la table `profiles`
 - SQL Editor: `SELECT role FROM profiles WHERE id = (SELECT id FROM auth.users WHERE email = 'votre@email.com');`
3. Si le role est `customer` , le promouvoir: `UPDATE profiles SET role = 'admin' WHERE id = ...`
4. Si la page affiche "Acces refuse" : le compte existe mais n'est pas admin
5. Si verrouillage (5 tentatives) : rafraichir la page et reessayer

"Le site est lent"

1. Verifier la taille des images dans Sanity (optimiser si > 2MB)
2. Verifier la region Supabase (choisir la plus proche des utilisateurs)
3. Activer le cache CDN sur Vercel (actif par default)
4. Verifier le nombre de produits charges par page

Fin du guide

Document cree en fevrier 2026. Mettre a jour en cas de changement majeur dans l'architecture ou les services utilises.